Dynamic analyzer

Olimpia Dejko
Natalia Bardadyn

październik 2026

1 Temat projektu

Projekt polega na stworzeniu aplikacji umożliwiającej przeprowadzanie podstawowej dynamicznej analizy oprogramowania. Aplikacja będzie uruchamiać przygotowane programy testowe, obserwować wybrane zdarzenia występujące podczas ich działania oraz prezentować zebrane informacje w czytelnym raporcie.

2 Cel projektu

Celem projektu jest opracowanie uproszczonego narzędzia demonstracyjnego, które pozwoli zapoznać się z praktycznymi metodami dynamicznej analizy oprogramowania. Aplikacja ma umożliwiać obserwowanie zachowania programu podczas jego wykonywania, zamiast ograniczać się wyłącznie do analizy jego kodu źródłowego. Projekt ma również pomóc w zrozumieniu, w jaki sposób można wykorzystywać informacje o procesach, operacjach na plikach i połączeniach sieciowych do analizy zachowania programów.

3 Główne funkcjonalności

Planowane funkcjonalności aplikacji:

1. **Wybór programu testowego** - wskazanie programu lub scenariusza przeznaczonego do analizy.
2. **Uruchomienie analizy** - rozpoczęcie kontrolowanego wykonania wybranego programu.
3. **Monitorowanie plików** - wykrywanie wybranych operacji, takich jak tworzenie i modyfikowanie plików w obserwowanym katalogu.
4. **Monitorowanie procesów** - zbieranie podstawowych informacji o analizowanym procesie oraz wybranych procesach potomnych.

5. **Monitorowanie sieci** - rejestrowanie kontrolowanych połączeń sieciowych wykonywanych przez program testowy.
6. **Prezentacja wyników** - wyświetlanie wykrytych zdarzeń w tabeli z informacją o czasie, kategorii i opisie zdarzenia.
7. **Generowanie raportu** - przygotowanie podsumowania przeprowadzonej analizy.
8. **Ocena ryzyka** - przypisywanie punktów wybranym zdarzeniom na podstawie jawnie zdefiniowanych reguł.

4 Technologie

Do realizacji projektu planujemy wykorzystać:

- **HTML** - strukturę interfejsu użytkownika.
- **CSS** - wygląd i układ aplikacji.
- **JavaScript** - obsługę interakcji użytkownika i aktualizację widoku.
- **Python** - implementację logiki analizatora, uruchamianie programów testowych i opracowywanie wyników.
  - **FastAPI** - nowoczesny, szybki framework webowy do budowy API.
  - **pefile** - biblioteka do parsowania i analizy struktur plików PE (Portable Executable), używana do odczytu nagłówków, sekcji i tabeli importów.
  - **Hashlib** - wbudowany moduł do obliczania kryptograficznych sum kontrolnych (MD5, SHA-256).
  - **UUID (Universally Unique Identifier)** - standard generowania unikalnych identyfikatorów używany do bezpiecznego przechowywania wgrywanych plików na serwerze.
- **Git i GitHub** - wersjonowanie kodu oraz współpracę nad projektem.

5 Architektura systemu i przepływ danych

Opis modelu klient-serwer, na którym opiera się aplikacja:

- **Backend (FastAPI/Python):** Odpowiada za odbieranie żądań HTTP, walidację strumienia plików, zarządzanie bezpiecznym katalogiem tymczasowym oraz parsowanie nagłówków PE za pomocą biblioteki pefile.
- **Frontend (HTML/CSS/JavaScript):** Odpowiada za interakcję z użytkownikiem, obsługę przeciągania plików (drag-and-drop), dynamiczne renderowanie tabeli zdarzeń, filtrowanie za pomocą zakładek oraz wizualizację postępu analizy.

- **Przepływ danych (Workflow):** Opis krokowy
użytkownik wskazuje plik .exe → frontend pakuje go do obiektu FormData i wysyła metodą POST do endpointu /api/analyze → backend generuje sumy kontrolne (MD5/SHA-256), bada tablicę importów i sekcje → zwraca sformatowaną tablicę zdarzeń w formacie JSON.

6 Napotkane błędy i sposoby ich rozwiązania

Podczas implementacji kolejnych funkcji projektu napotkano na kilka wyzwań związanych ze stanem interfejsu (UI) oraz synchronizacją kodu. Poniżej zebrano najważniejsze z nich wraz z zastosowanymi poprawkami:

1. **Przeciążenie serwera przez pliki o niestandardowym rozmiarze**
Opis: Przesłanie skrajnie dużego pliku mogłoby doprowadzić do wyczerpania pamięci RAM serwera lub nadmiernego obciążenia procesora podczas parsowania struktury PE.
Rozwiązanie: Wprowadzono restrykcyjną walidację rozmiaru przesyłanego strumienia danych po stronie backendu (maksymalnie 150 MB), odrzucającą nieodpowiednie pliki na samym początku żądania HTTP.

2. **Znikanie przycisku wyboru pliku po błędnej walidacji lub niepowodzeniu analizy**
Opis: Gdy użytkownik przesłał plik niebędący plikiem .exe lub gdy wystąpił błąd komunikacji z serwerem, kluczowe elementy interfejsu (takie jak przycisk wyboru/zmiany pliku) znikały, pozostawiając interfejs w zablokowanym stanie.
Rozwiązanie: Zmodyfikowano logikę w pliku app.js, rozdzielając przycisk wyboru pliku od stanu analizy. Zapewniono, że przycisk wyboru pliku jest zawsze widoczny, niezależnie od wystąpienia błędu czy statusu przesłanego pliku.

3. **Zagrożenie związane ze znakami specjalnymi w nazwach plików**
Opis: Użytkownicy mogli przesyłać pliki zawierające spacje, polskie znaki diakrytyczne lub znaki specjalne, co na backendzie mogłoby prowadzić do błędów ścieżek lub podatności bezpieczeństwa.
Rozwiązanie: Zastosowano generowanie losowych, unikalnych nazw tymczasowych przy użyciu standardu UUID dla każdego wgrywanego pliku. Dzięki temu operacje plikowe na serwerze są przewidywalne i bezpieczne.

4. **Surowe powiadomienia systemowe zaburzające estetykę**
Opis: Komunikaty o błędach (np. nieprawidłowe rozszerzenie pliku lub funkcja w budowie) były obsługiwane przez standardowe okienka alert(), co psuło spójność ciemnego, nowoczesnego motywu aplikacji.
Rozwiązanie: Zaimplementowano dedykowany komponent baneru powiadomień stylizowany w CSS na wzór powiadomień typu toast. Dodano węższą, wyśrodkowaną formę oraz płynne animacje pojawiania się i wygaszania,

dzięki czemu błędy znikają samoczynnie po kilku sekundach lub po kliknięciu krzyżyka.

7 Bezpieczeństwo, walidacja i architektura proaktywna

Opis mechanizmów chroniących aplikację i serwer przed niepożądanym zachowaniem:

- **Walidacja rozszerzeń i typów:** Restrykcyjne sprawdzenie, czy przesyłany plik posiada rozszerzenie .exe zarówno po stronie klienta, jak i serwera.
- **Ochrona przed Path Traversal i kolizjami nazw:** Zastosowanie losowych identyfikatorów UUID do nadawania nazw plikom w folderze roboczym serwera, co uniezależnia system od znaków specjalnych i spacji w oryginalnej nazwie pliku.
- **Kontrola rozmiaru danych:** Wprowadzenie twardego limitu rozmiaru pliku (maksymalnie 150 MB), zapobiegającego wyczerpaniu pamięci RAM serwera.

8 Plan realizacji

Harmonogram prac obejmuje następujące etapy:

1. Przygotowanie interfejsu HTML i CSS oraz podstawowej obsługi przycisków.
2. Implementacja backendu w Pythonie i połączenie go z interfejsem.
3. Przygotowanie bezpiecznych programów testowych.
4. Implementacja pierwszego modułu monitorowania i prezentacji wykrytych zdarzeń.
5. Rozbudowa o kolejne moduły monitorowania.
6. Opracowanie raportowania i ewentualnej oceny ryzyka.
7. Przeprowadzenie testów, przygotowanie dokumentacji końcowej i prezentacji projektu.

9 Oczekiwany rezultat

Rezultatem projektu ma być działająca aplikacja z interfejsem graficznym, która umożliwi uruchamianie przygotowanych programów testowych, rejestrowanie wybranych zdarzeń podczas ich działania oraz prezentowanie wyników w postaci czytelnego raportu. Projekt ma również umożliwić praktyczne zaprezentowanie podstaw dynamicznej analizy oprogramowania i sposobów wykorzystania informacji o zachowaniu programu w kontekście cyberbezpieczeństwa.

Spis treści

1 Temat projektu **1**

2 Cel projektu **1**

3 Główne funkcjonalności **1**

4 Technologie **2**

5 Architektura systemu i przepływ danych **2**

6 Napotkane błędy i sposoby ich rozwiązania **3**

7 Bezpieczeństwo, walidacja i architektura proaktywna **4**

8 Plan realizacji **4**

9 Oczekiwany rezultat **4**